■ Mushroom Business Management & E-Commerce Platform (Mushroom ERP)

Document 02: Full-Stack System Architecture & Technical Specification

Project Name: SporeERP
Document Code: DOC-02-ARCH
Version: 1.0.0-PROD
Target Stack: Next.js 15+ (App Router) * TypeScript * MariaDB 11.x * Prisma ORM * NextAuth.js v5 * Redis

1. High-Level Architecture Overview

SporeERP employs a **Modern Monolithic Full-Stack Architecture** running within Next.js. This approach maximizes development velocity for a 2-founder team, minimizes DevOps maintenance overhead, and ensures strict type-safety across frontend components, API endpoints, business domain services, and database queries.

2. Directory Structure & Modular Layout

The codebase strictly separates presentation, domain logic, data access, and utilities:

3. Layered Service Architecture Pattern

To prevent architectural degradation, business rules are never placed directly inside React components or API routes. Every transaction flows through a strict **4-Tier Pipeline**:

4. Transactional Concurrency Control & Stock Safety

Mushrooms are highly perishable and sold across multiple channels simultaneously. To eliminate race conditions and double-booking, stock reservation utilizes **Atomic MariaDB Transactions with Pessimistic Locking**:

5. Security & RBAC Middleware Architecture

Every incoming request to the application passes through Next.js Edge Middleware before executing any route handler:

---

6. Performance, Caching & Scalability Controls

1. **Incremental Static Regeneration (ISR):** Public catalog pages (`/products`, `/products/[slug]`) are statically generated and revalidated every **60 seconds**, guaranteeing sub-50ms page loads.
2. **On-Demand Cache Tag Revalidation:** When an admin modifies product prices or stock status, Next.js `revalidateTag('product-catalog')` instantly purges stale cache globally.
3. **Database Connection Pooling:** Managed via Prisma connection pool configuration optimized for MariaDB:
   - `connection_limit = 15` (Max simultaneous active connections per container).
   - `pool_timeout = 20` (Seconds before timing out a waiting query).
4. **Redis Integration:** Used for NextAuth session caching, rate-limiting checkout attempts (max 5 requests/minute per IP), and pub/sub real-time dashboard notifications.